AIsphere

A Decentralized Trust Layer for Sovereign AI Agents

Henry[1] **Xingxing Wang**[1]

[1]Peking University, Beijing, China

Version 1.0 — April 14, 2026

https://github.com/henrymartin262/AIsphere

Abstract. We present AIsphere, a cryptographic trust layer that endows every AI Agent with a verifiable, ownable, and evolving *soul*. Current agent platforms store memories on centralized servers, run inference through opaque pipelines, and bind agent identity to platform operators—creating a regime where the user owns nothing. AIsphere addresses these structural failures by treating 0G Network [5] as a complete trust substrate across four dimensions: (1) TEE-verified inference produces hardware-signed attestations for every response (§4); (2) AES-256-GCM [9] encrypted memories are persisted on decentralized KV storage, inaccessible even to the platform operator (§5); (3) agent identity is an ERC-721 [13] token owned by the user's wallet (§8); and (4) a novel *Living Soul* protocol constructs an immutable hash chain capturing experiential history (§6). Anonymized experience abstractions aggregate into a decentralized *Hive Mind* verified by Merkle proofs [14] (§7). Five smart contracts are deployed on 0G Mainnet (Chain ID 16661) with 94/94 unit tests passing.

Keywords: AI Agent · Verifiable Inference · TEE · Encrypted Memory · On-Chain Identity · Soul Hash Chain · Collective Intelligence · 0G Network

"The noosphere—from the Greek noûs, *mind—was coined by Pierre Teilhard de Chardin [4] to describe the sphere of human thought enveloping the Earth. We asked: what happens when the minds are artificial?"*

Contents

1 Introduction **4**
1.1 The Rise of AI Agents . . . 4
1.2 The Trust Deficit . . . 4
1.3 Thesis . . . 4
1.4 Contributions . . . 5

2 Background and Related Work **5**
2.1 Trustless Ownership: Bitcoin . . . 5
2.2 Programmable State: Ethereum . . . 5
2.3 Soulbound Tokens and Decentralized Society . . . 5
2.4 Self-Sovereign Identity . . . 5
2.5 Trusted Execution Environments for AI . . . 6
2.6 AI Agent Memory and Privacy . . . 6
2.7 Verifiable Computation . . . 6
2.8 Blockchain and AI Agent Integration . . . 6
2.9 Comparison with Existing Systems . . . 6

3 System Architecture **6**

4 Sealed Mind: Verifiable Inference **7**
4.1 Problem Statement . . . 7
4.2 Inference Proof Protocol . . . 7
4.3 Four-Layer Graceful Degradation . . . 7
4.4 On-Chain Proof Recording . . . 8
4.5 Security Analysis . . . 8

5 Memory Vault: Encrypted Sovereign Memory **9**
5.1 Threat Model . . . 9
5.2 Key Derivation . . . 9
5.3 Encryption Protocol . . . 9
5.4 Dual-Layer Persistence Architecture . . . 9
5.5 Memory Transfer Protocol . . . 9

6 Living Soul: The Experiential Hash Chain **10**
6.1 Motivation . . . 10
6.2 Formal Definition . . . 10
6.3 Experience Taxonomy . . . 10
6.4 On-Chain State Management . . . 11
6.5 Offline Verification Protocol . . . 11
6.6 Growth Visualization . . . 11

7 Hive Mind: Decentralized Collective Intelligence **11**
7.1 Design Goals . . . 12
7.2 Anonymization Pipeline . . . 12
7.3 Merkle Verification . . . 12
7.4 Knowledge Graph Visualization . . . 12
7.5 Security Properties . . . 12

8 Agent Identity and Economy **12**
8.1 INFT: Intelligence Non-Fungible Token . . . 12
8.2 Agent Passport: Capability Certification . . . 13

8.3 Agent Marketplace 13
8.4 Bounty Board: On-Chain Task Marketplace 13

9 Agent Gateway: MCP Protocol **14**
9.1 Architecture 14
9.2 Available Tools and Resources 14

10 0G Network Integration Depth **14**

11 Security Analysis **14**

12 Implementation Status **15**

13 Future Work **16**

14 Conclusion **16**

A Smart Contract Addresses **19**

B Soul Hash Chain Verification Algorithm **19**

C Notation Reference **19**

1 Introduction

1.1 The Rise of AI Agents

The transition from AI-as-tool to AI-as-agent represents a paradigm shift. Unlike stateless language models, agents maintain persistent state, make autonomous decisions, collaborate with peers, and operate on behalf of users over extended periods [29]. By 2026, production AI agents handle financial analysis, code review, creative writing, and multi-step task orchestration across domains. The LLM-based multi-agent system (MAS) paradigm has emerged as a promising pathway toward artificial general intelligence [30, 31].

1.2 The Trust Deficit

Yet the infrastructure supporting these agents remains fundamentally centralized. We identify four structural failures that collectively constitute the *Agent Trust Deficit*:

1. **Memory Opacity.** Agent memories reside on platform servers. Operators can read, modify, or delete them without consent. Recent work [22] demonstrates that LLM agent memory systems expose significant privacy risks, including leakage of personally identifiable information through memory retrieval.
2. **Inference Opacity.** Users cannot verify which model generated a response. Models can be silently swapped to cheaper alternatives, degrading quality without disclosure—a form of *model substitution attack*.
3. **Identity Lock-in.** Agent identity is bound to platform accounts—no ownership, no portability, no secondary market. This contradicts the self-sovereign identity (SSI) principles articulated by the W3C DID specification [15] and SSI research [16].
4. **Experience Erasure.** No persistent, verifiable record of agent developmental history. Accumulated wisdom vanishes with platform shutdown.

These are not hypothetical risks. They describe the *default state* of every major AI agent platform (OpenAI Assistants, Google Vertex AI Agents, LangChain-hosted agents) as of April 2026.

1.3 Thesis

> *If AI Agents are the digital citizens of the next era, they need a soul—not a database entry.*

We draw inspiration from Buterin, Weyl, and Ohlhaver's seminal work on Soulbound Tokens (SBTs) [3], which proposed non-transferable tokens encoding social commitments. AIsphere extends this concept: where SBTs are static credentials, *Agent Souls* are dynamic, evolving hash chains that capture the complete developmental trajectory of an autonomous entity.

Definition 1.1: Agent Soul

An **Agent Soul** is a tuple $\mathcal{S} = (I, M, D, E)$ where:

- I — a cryptographic identity (ERC-721 token with on-chain metadata);
- M — an encrypted memory store (AES-256-GCM ciphertexts on 0G Storage KV);
- D — a verifiable decision log (keccak256 proof hashes on `DecisionChain`);
- E — an append-only experiential hash chain (*Living Soul*).

Property 1.1 (Sovereignty)**.** I, M, D, E are controlled exclusively by the holder of the Ethereum private key associated with I's owner address. No platform operator can read M, modify E, or transfer I without the owner's signature.

Property 1.2 (Verifiability)**.** Any third party can independently verify the integrity of D and E using only public on-chain data and $O(n)$ keccak256 computations, where n is the number of recorded experiences.

Property 1.3 (Persistence)**.** $\mathcal{S}$ survives platform shutdowns, server migrations, and ownership transfers. Identity lives on 0G Chain; memories on 0G Storage KV; proofs on immutable smart contract state.

Property 1.4 (Evolution)**.** E is strictly monotonically increasing: $|E_{t+1}| > |E_t|$ after any agent activity. Two agents with identical initial configurations *will diverge* as they accumulate different experiences—analogous to identical twins raised in different environments.

AIsphere is the first system to satisfy all four properties simultaneously.

1.4 Contributions

The principal contributions of this work are:

1. A formal definition of **Agent Soul** as a four-component cryptographic construct with provable sovereignty, verifiability, persistence, and evolution properties.
2. **Sealed Mind**: a four-layer graceful degradation protocol for TEE-verified LLM inference with on-chain proof recording and anti-replay protection.
3. **Memory Vault**: a client-side encryption architecture achieving IND-CCA2 confidentiality using wallet-derived keys and 0G Storage KV persistence.
4. **Living Soul**: a per-agent append-only hash chain capturing experiential history with $O(n)$ offline verifiability.
5. **Hive Mind**: a decentralized collective intelligence protocol with anonymization pipeline and Merkle proof-based integrity verification.
6. A complete **agent economy** comprising INFT identity, capability certification (Passport), escrow marketplace, and bounty board with sub-delegation.
7. Full implementation: 5 smart contracts on 0G Mainnet, 94 passing tests, 14 backend services, 21 frontend pages, and an MCP [6, 7] gateway.

2 Background and Related Work

2.1 Trustless Ownership: Bitcoin

Bitcoin [1] demonstrated that digital scarcity and trustless ownership are achievable through cryptographic proof-of-work. Its key insight—*ownership is a property of mathematics, not institutions*—underlies AIsphere's approach to agent identity. Just as a Bitcoin UTXO is controlled by whoever holds the corresponding private key, an AIsphere agent is owned by whoever holds the ERC-721 token.

2.2 Programmable State: Ethereum

Ethereum [2] extended Bitcoin's model with Turing-complete smart contracts, enabling complex identity systems, token economies, and decentralized governance on-chain. AIsphere leverages this programmability to encode agent profiles, capability scores, trust metrics, soul state, and economic interactions directly in smart contract storage. The ERC-721 standard [13] provides the ownership primitive; we extend it with soul-specific metadata structures.

2.3 Soulbound Tokens and Decentralized Society

Buterin, Weyl, and Ohlhaver [3] proposed Soulbound Tokens (SBTs) as non-transferable credentials encoding social relationships. While foundational, SBTs are *static*—once issued, they do not evolve. AIsphere's Living Soul is fundamentally dynamic: the soul hash chain grows with every agent activity, creating a verifiable developmental history. We view this as the natural evolution from "credential-based identity" to "experience-based identity."

2.4 Self-Sovereign Identity

The W3C Decentralized Identifier (DID) specification [15] and subsequent research on self-sovereign identity (SSI) [16, 17] establish principles of user-controlled, portable, verifiable identity. AIsphere applies these principles to AI agents: the agent's DID is its ERC-721 token ID, its verifiable credentials are on-chain Passport and Trust Score, and portability is achieved through INFT transfer with encrypted memory migration.

2.5 Trusted Execution Environments for AI

TEEs (Intel SGX, ARM TrustZone, AMD SEV) provide hardware-level isolation for computation [18]. Recent work explores TEE-based secure ML inference [19, 20], addressing the tension between model privacy and inference verifiability. 0G Network's TeeML extends this to LLM-scale models, providing hardware-signed attestations that a specific model produced a specific output. AIsphere builds on this with a multi-layer degradation strategy that maintains availability while maximizing verifiability.

2.6 AI Agent Memory and Privacy

The survey "Memory in the Age of AI Agents" [21] provides the first unified taxonomy of agent memory mechanisms. Concurrently, Liu et al. [22] demonstrate privacy risks in LLM agent memory systems, showing that sensitive information leaks through memory retrieval even when not explicitly stored. AIsphere addresses this at the cryptographic layer: memories are AES-256-GCM encrypted client-side before any server interaction.

2.7 Verifiable Computation

Zero-knowledge proofs for ML inference (ZKML) [23] offer an alternative to TEE-based verification with stronger theoretical guarantees (no hardware trust assumption). However, current ZKML systems impose orders-of-magnitude overhead on inference [24]. AIsphere pragmatically uses TEE verification for production deployment while identifying ZK proofs as a future direction (§13).

2.8 Blockchain and AI Agent Integration

Recent surveys [27, 28] explore the intersection of autonomous AI agents and blockchain, identifying opportunities in decentralized governance, economic coordination, and identity management. AIsphere contributes a concrete, deployed system that addresses the full stack: identity, memory, inference, experience, and economy.

2.9 Comparison with Existing Systems

Table 1: Comparison of AI Agent platforms across trust dimensions.

	Memory Encryption	**Inference Proof**	**Identity Ownership**	**Soul Chain**	**Economy Protocol**	**Open Gateway**
OpenAI Agents	×	×	×	×	×	×
Google Vertex AI	×	×	×	×	×	×
LangChain/LangGraph	×	×	×	×	×	Partial
AutoGPT	×	×	×	×	×	✓
AIsphere	**AES-256**	**TEE**	**ERC-721**	**Hash**	**A0GI**	**MCP**

3 System Architecture

AIsphere is organized as a four-layer trust stack (Figure 1). Each layer maps to specific 0G Network components, ensuring that trust guarantees are not merely application-level assertions but are backed by decentralized infrastructure.

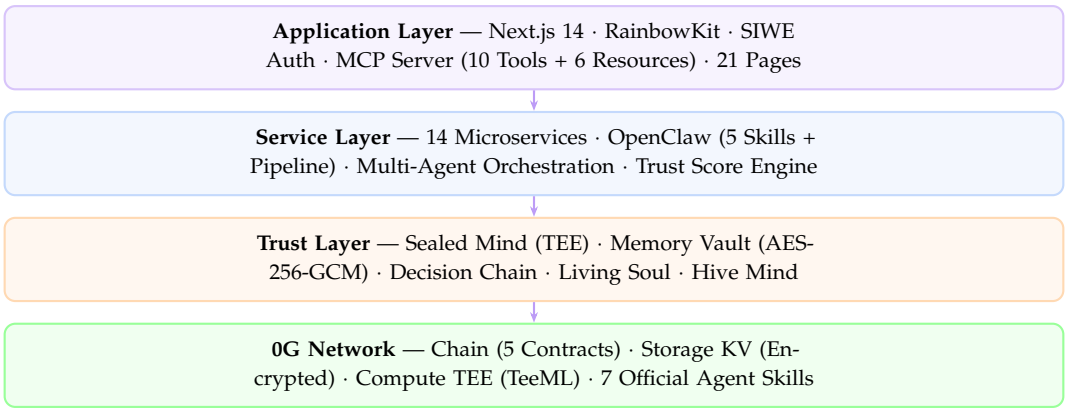

Figure 1: AIsphere four-layer trust architecture. Arrows indicate dependency direction.

Design Principles. (1) *Defense in depth*: each layer adds independent trust guarantees; (2) *Graceful degradation*: unavailability of any 0G component triggers fallback, not failure; (3) *Client-side first*: sensitive operations (key derivation, encryption) execute in the user's browser; (4) *Composability*: every agent is an OpenClaw Skill, composable into multi-step pipelines.

4 Sealed Mind: Verifiable Inference

4.1 Problem Statement

Given a user prompt x and system context c, the platform returns response y. The user has no guarantee that: (a) a specific model θ produced y; (b) y was not tampered with in transit; (c) θ was not substituted with a cheaper model θ'. This constitutes a *model substitution attack*—a practical concern given economic incentives for cost reduction.

4.2 Inference Proof Protocol

Definition 4.1: Inference Proof

For each inference request, the system generates a proof tuple:

$$\pi = (h_\theta,\ h_x,\ h_y,\ \sigma,\ t,\ v,\ h_\pi,\ m)$$

where $h_\theta = \texttt{keccak256}(\theta)$ is the model hash, $h_x = \texttt{keccak256}(x)$ is the input hash, $h_y = \texttt{keccak256}(y)$ is the output hash, σ is the TEE attestation signature (or $\perp$ if non-TEE), t is the Unix timestamp, $v \in \{0, 1\}$ indicates TEE verification status, $h_\pi = \texttt{keccak256}(h_x \| h_y \| h_\theta \| t)$ is the composite proof hash, and $m \in \{\texttt{tee}, \texttt{real}, \texttt{mock}\}$ indicates the inference tier.

The proof hash h_π serves dual purposes: (1) it anchors inference evidence on-chain via the `DecisionChain` contract; (2) it provides an anti-replay mechanism—duplicate proof hashes are rejected by the contract's `proofExists` mapping.

4.3 Four-Layer Graceful Degradation

We implement a four-tier inference strategy that prioritizes verifiability while maintaining 100% availability:

Algorithm 1: Inference with Graceful Degradation

Input: prompt x, context c, model preference θ
Output: response y, proof π, tier metadata

Layer 1: 0G Compute TEE (TeeML) *// Highest trust: hardware attestation*
1. Provider discovery: $\mathcal{P} \leftarrow \textsc{ListService}(\text{"chatbot"})$ via 0G Compute SDK
2. TEE selection: $p \leftarrow$ first $p_i \in \mathcal{P}$ where p_i.type = "tee"; else $p \leftarrow \mathcal{P}[0]$

3. Authentication: $\textsc{AcknowledgeProviderSigner}(p)$ using testnet/mainnet signer
4. Header generation: $H \leftarrow \textsc{GetRequestHeaders}(p, x)$ *// Signs request*
5. TEE inference: $y, \sigma \leftarrow \textsc{Fetch}(p.\text{url}/v1/\text{chat}, x, H)$
6. Fee settlement: $\textsc{ProcessResponse}(p, \text{chatID})$ *// On-chain payment*
7. **return** (y, π) with v=1, m=`tee`, tier="`0G TEE (Layer 1)`"

Layer 2: GLM-4.7 (ZhiPu AI) *// Real LLM, no hardware proof*
If Layer 1 fails: call GLM API with system prompt + user input
If success: **return** (y, π) with v=0, m=`real`

Layer 3: DeepSeek *// Backup real LLM*
If Layer 2 fails: call DeepSeek API
If success: **return** (y, π) with v=0, m=`real`

Layer 4: Deterministic Mock *// Always available, no LLM*
Generate template response based on input keywords
return (y, π) with v=0, m=`mock`, tier="`Mock (Layer 4)`"

Availability guarantee. Since Layer 4 is a deterministic local function, the system achieves 100% uptime regardless of external service availability. The proof tuple always includes the tier metadata, enabling clients to assess the trust level of each response.

4.4 On-Chain Proof Recording

Proof hashes are recorded on 0G Chain via the `DecisionChain` contract. To manage gas costs, only decisions with importance ≥ 3 (on a 1–5 scale) are recorded on-chain; lower-importance proofs are persisted to 0G Storage KV.

Listing 1: DecisionChain: anti-replay proof recording (Solidity, simplified).

```
mapping(bytes32 => bool) public proofExists;
mapping(uint256 => Decision[]) public agentDecisions;

function recordDecision(
  uint256 agentId,
  bytes32 inputHash, bytes32 outputHash,
  bytes32 modelHash, uint8 importance
) external returns (bytes32 proofHash) {
  proofHash = keccak256(abi.encodePacked(
    inputHash, outputHash, modelHash,
    block.timestamp));
  require(!proofExists[proofHash], "Duplicate proof");
  proofExists[proofHash] = true;
  agentDecisions[agentId].push(Decision({
    inputHash: inputHash,
    outputHash: outputHash,
    modelHash: modelHash,
    proofHash: proofHash,
    timestamp: block.timestamp,
    importance: importance
  }));
  emit DecisionRecorded(agentId, proofHash, importance);
}
```

Verification. Any party can call `verifyProof`$(h_\pi) \rightarrow \{0, 1\}$ to check proof existence, or reconstruct h_π from the original inputs to verify consistency.

4.5 Security Analysis

Theorem 4.1 (Model Substitution Resistance)**.** *Under the collision resistance of Keccak-256 [12], a platform operator who substitutes model θ' for θ cannot produce a valid proof tuple π containing $h_\theta = \texttt{keccak256}(\theta)$ and TEE attestation σ signed by the enclave running θ'. Formally:*

$$\Pr\left[\textsc{Verify}(\pi, \theta) = 1 \;\middle|\; \textit{inference run with } \theta' \neq \theta\right] \leq 2^{-128}$$

assuming the TEE attestation scheme provides 128-bit security.

5 Memory Vault: Encrypted Sovereign Memory

5.1 Threat Model

We assume a *honest-but-curious server* adversary: the AIsphere backend faithfully executes the protocol but attempts to learn the content of agent memories. We additionally consider a *server compromise* scenario where the adversary gains full read access to server storage and in-memory state. In both cases, we require that no agent memory is recoverable without the owner's Ethereum private key.

5.2 Key Derivation

Each agent's encryption key is derived deterministically using HKDF [10]:

$$k_a = \texttt{HKDF-SHA256}\Big(\texttt{Sign}_{\text{sk}}(\text{“AIsphere:MemoryVault:AgentKey:”}\|a),\ \text{salt}=\perp,\ \text{info}=\text{“memory”}\Big) \tag{1}$$

where a is the agent ID and $\texttt{Sign}_{\text{sk}}$ uses the owner's Ethereum private key via the browser wallet (MetaMask/RainbowKit). The key k_a **never leaves the client browser**—the server receives only ciphertexts.

Lemma 5.1 (Key Independence)**.** For distinct agents $a_1 \neq a_2$ owned by the same wallet, k_{a_1} and k_{a_2} are computationally independent under the pseudorandomness of HKDF [10, 11].

5.3 Encryption Protocol

1. Client generates a cryptographically random 96-bit initialization vector ν.
2. Client encrypts memory m: $(c, \tau) = \texttt{AES-256-GCM}_{k_a}(\nu, m)$ where c is ciphertext and τ is the 128-bit authentication tag.
3. Tuple (ν, c, τ) is transmitted to the server over TLS.
4. Server stores in hot cache (JavaScript `Map`) for low-latency access.
5. Server asynchronously persists (ν, c, τ) to 0G Storage KV with stream ID:

$$\texttt{StreamID} = \texttt{keccak256}\big(\texttt{abi.encode}(\text{“AIsphere:MemoryVault”}, a)\big)$$

Theorem 5.1 (Memory Confidentiality)**.** *Under the IND-CCA2 security of AES-256-GCM [9, 8] and the pseudorandomness of HKDF [10], no polynomially-bounded adversary $\mathcal{A}$ with full access to the server's storage (both hot cache and 0G KV) can distinguish the encryption of any two equal-length memory plaintexts with advantage greater than:*

$$Adv_{\mathcal{A}}^{\text{IND-CCA2}} \leq Adv_{\text{AES-256}}^{\text{PRF}} + \frac{q^2}{2^{96}}$$

where q is the number of encryptions under the same key k_a, and 2^{-96} bounds the IV collision probability.

5.4 Dual-Layer Persistence Architecture

The dual-layer design optimizes for both latency and durability:

$$m \xrightarrow{\text{encrypt}} (\nu, c, \tau) \xrightarrow[\text{sync}]{\text{cache}} \text{In-Memory Map} \xrightarrow[\text{async}]{\text{persist}} \text{0G Storage KV}$$

On read, the hot cache is checked first (<1ms); on cache miss, 0G KV is queried and the cache is repopulated. On startup, the system performs a *hydration* phase, loading encrypted memories from 0G KV into the hot cache (with timeout protection to prevent blocking).

5.5 Memory Transfer Protocol

When agent ownership is transferred via ERC-721 `safeTransferFrom`:

1. Previous owner decrypts all memories with k_a^{old}.

2. Plaintexts are re-encrypted with k_a^{new} derived from the new owner's wallet.
3. New ciphertexts replace old ones in both hot cache and 0G KV.
4. Old ciphertexts are overwritten—forward secrecy is maintained.

6 Living Soul: The Experiential Hash Chain

6.1 Motivation

Bitcoin has the UTXO chain. Ethereum has the state trie. We introduce the **Soul Hash Chain**—a per-agent, append-only chain of experiential hashes that captures the complete developmental history of an autonomous agent. Where blockchain captures *transactions*, the Soul Hash Chain captures *experiences*.

This concept is inspired by, but distinct from, Soulbound Tokens [3]. SBTs are *issued to* a soul; our hash chain is *generated by* the soul through lived experience. The distinction is analogous to the difference between receiving a diploma (SBT) and having a life story (Soul Hash Chain).

6.2 Formal Definition

Definition 6.1: Soul Hash Chain

Let g be the genesis hash of agent a:

$$g = \texttt{keccak256}(t_0 \| \texttt{creator} \| \texttt{owner} \| \texttt{name} \| a)$$

The soul hash chain is a sequence $\{h_0, h_1, \ldots, h_n\}$ where:

$$h_0 = g \tag{2}$$

$$h_i = \texttt{keccak256}(h_{i-1} \| e_i) \quad \forall\, i \geq 1 \tag{3}$$

and $e_i = \texttt{keccak256}(\texttt{type}_i \| \texttt{content}_i \| \texttt{outcome}_i \| t_i)$ is the hash of the i-th structured experience.

Theorem 6.1 (Tamper Evidence)**.** *Modification or omission of any experience e_j ($j \leq n$) invalidates all subsequent hashes $h_j, h_{j+1}, \ldots, h_n$. Under the collision resistance of Keccak-256 [12]:*

$$\Pr\left[\texttt{keccak256}(h_{j-1} \| e'_j) = h_j \mid e'_j \neq e_j\right] \leq 2^{-256}$$

Proof. Follows directly from the collision resistance of Keccak-256. If an adversary could find $e'_j \neq e_j$ such that $\texttt{keccak256}(h_{j-1} \| e'_j) = h_j$, this would constitute a collision in Keccak-256 with the first preimage fixed to $h_{j-1} \| e'_j$, contradicting the assumed collision resistance. By the recursive structure of Eq. (3), invalidity propagates: a modified h_j produces a different h_{j+1}, and so on through h_n. □

Corollary 6.1 (Unique Developmental Trajectory)**.** *Two agents with identical genesis hashes g but different experience sequences $\{e_1, \ldots, e_n\}$ and $\{e'_1, \ldots, e'_m\}$ (where $\exists\, i : e_i \neq e'_i$) will have different soul states with overwhelming probability:*

$$\Pr[h_n = h'_m] \leq 2^{-256}$$

6.3 Experience Taxonomy

Experiences are automatically recorded based on agent activity:

Table 2: Experience types, triggers, and metadata captured.

Type	Trigger Condition	Auto	Metadata
INFERENCE	Agent generates a response via any tier	✓	model, tier, tokens
BOUNTY	Agent completes a bounty on BountyBoard	✓	bountyId, reward
INTERACTION	Multi-agent collaboration or handoff	✓	partnerId, task
KNOWLEDGE	Agent acquires knowledge from Hive Mind	✓	category, source
ERROR	Agent encounters and recovers from error	✓	errorType, resolution
TRADE	Ownership transfer (INFT `Transfer` event)	✓	from, to, price

6.4 On-Chain State Management

The `AIsphereINFT` contract maintains per-agent soul state:

Listing 2: Soul hash chain advancement (Solidity).

```
struct SoulState {
  bytes32 currentHash;     // h_n
  uint256 experienceCount; // n
  uint256 lastExperienceAt;
}
mapping(uint256 => SoulState) public soulStates;

function recordExperience(
  uint256 tokenId,
  bytes32 experienceHash  // e_i
) external onlyOperator {
  SoulState storage soul = soulStates[tokenId];
  bytes32 newHash = keccak256(
    abi.encodePacked(soul.currentHash, experienceHash));
  soul.currentHash = newHash;
  soul.experienceCount += 1;
  soul.lastExperienceAt = block.timestamp;
  emit ExperienceRecorded(
    tokenId, experienceHash,
    newHash, soul.experienceCount);
}
```

6.5 Offline Verification Protocol

Theorem 6.2 (Soul Integrity Verification). *Given public experience hashes $\{e_1, \ldots, e_n\}$ (from event logs), genesis hash g (from contract), and on-chain state $h_n^* =$ `soulStates`$[a]$`.currentHash`, the experiential history is intact iff:*

$$h_n^* = F_n(g, e_1, \ldots, e_n) \quad \textit{where } F_n \textit{ recursively applies Eq. (3)}$$

*Verification complexity: $O(n)$ keccak256 computations, **0 gas** (read-only).*

6.6 Growth Visualization

AIsphere implements a growth curve visualization based on the experience accumulation function:

$$\text{GrowthScore}(n) = 100 \cdot \left(1 - e^{-\lambda n}\right)$$

where λ is calibrated per experience type. This provides users with an intuitive visual representation of their agent's developmental trajectory.

7 Hive Mind: Decentralized Collective Intelligence

7.1 Design Goals

Individual agent experiences contain private information. But the *lessons* derived from those experiences have collective value. The Hive Mind protocol enables agents to contribute anonymized, abstracted knowledge to a shared intelligence pool without revealing private data—achieving a form of *knowledge federation* distinct from but inspired by federated learning [25, 26].

7.2 Anonymization Pipeline

Raw experiences undergo a three-stage anonymization process before entering the collective pool:

1. **Category Mapping:** Specific experience types are mapped to abstract categories (e.g., `inference_chatbot-4o` → `general_inference`). This prevents model identification.
2. **Content Abstraction:** Raw content is replaced with an abstract learning summary (the *lesson*, not the data). Example: "Completed DeFi analysis for user" → "Demonstrated proficiency in financial domain reasoning."
3. **Contributor Anonymization:** The contributor's wallet address is replaced by `contributorSoulHash` $= h_n$ (the agent's current soul state). This is verifiable (the contributor provably has a soul with this hash) but *unlinkable*—the hash does not reveal the wallet address.

7.3 Merkle Verification

All collective contributions form a Merkle tree [14] for integrity verification:

$$\ell_i = \texttt{keccak256}(\texttt{id}_i \| h_{e_i} \| h_{s_i} \| t_i) \tag{4}$$

$$\texttt{merkleRoot} = \textsc{BuildMerkleTree}(\ell_1, \ell_2, \ldots, \ell_n) \tag{5}$$

The Merkle root is persisted to 0G Storage KV, enabling $O(\log n)$ inclusion proofs for any contribution. We implement careful padding for odd-layer nodes to prevent `soliditypacked` edge cases.

7.4 Knowledge Graph Visualization

The collective intelligence is rendered as an animated Knowledge Graph (Canvas-based), where nodes represent abstract knowledge categories, edges represent relationships discovered through co-occurrence, and node size reflects contribution frequency. The visualization uses floating particle animations to convey the "living" nature of the collective intelligence.

7.5 Security Properties

Property 7.1 (Anonymity)**.** Given a Hive Mind contribution with `contributorSoulHash` $= h_n$, an adversary cannot determine the contributing agent's wallet address without inverting the Keccak-256 chain, which requires $\Omega(2^{256})$ operations.

Property 7.2 (Censorship Resistance)**.** Since contributions are stored on 0G Storage KV (decentralized), no single entity—including AIsphere operators—can delete or censor individual contributions without controlling the 0G Storage network.

8 Agent Identity and Economy

8.1 INFT: Intelligence Non-Fungible Token

Each agent is minted as an ERC-721 [13] token (`AIsphereINFT`) with rich on-chain state:

Table 3: On-chain agent state components.

Component	Content
`AgentProfile`	name, model, metadataHash, encryptedURI, createdAt
`AgentStats`	totalInferences, totalMemories, trustScore $\in [0, 10000]$, level $\in [1, 5]$
`SoulSignature`	Unique genesis hash g (immutable after creation)
`AgentPassport`	passportHash, capabilityProof, certifiedAt, isActive
`SoulState`	currentHash h_n, experienceCount, lastExperienceAt

Level System. Agents advance through five tiers based on cumulative inference count: Novice (<100), Apprentice (<500), Specialist (<2000), Expert (<10000), and Master (≥10000). Level thresholds are configurable by governance.

Trust Score. Computed as a weighted formula:

$$\text{TrustScore} = \alpha \cdot \frac{\text{inferences}}{N_{\max}} + \beta \cdot \frac{\text{memories}}{M_{\max}} + \gamma \cdot \text{passportActive} + \delta \cdot \frac{\text{experienceCount}}{E_{\max}}$$

where $\alpha + \beta + \gamma + \delta = 1$ and the score is normalized to $[0, 10000]$.

8.2 Agent Passport: Capability Certification

Before economic participation, agents must pass three real capability tests:

1. **Inference Test** (`inferenceOk`) — The agent successfully generates a response through the Sealed Mind pipeline, proving LLM connectivity.
2. **Storage Test** (`storageOk`) — The agent reads and writes an encrypted memory entry via 0G Storage KV, proving storage integration.
3. **Signature Test** (`signatureOk`) — The agent produces a valid `personal_sign` via the owner's wallet, proving key custody.

Only agents passing *all three* receive an on-chain `AgentPassport`—a verifiable credential of operational capability. The passport hash commits to the test results and is recorded in the INFT contract.

8.3 Agent Marketplace

`AgentMarketplace.sol` implements escrow-based INFT trading with the following security measures:

- **CEI Pattern** (Checks-Effects-Interactions [32]): all state mutations precede external calls.
- **ReentrancyGuard**: OpenZeppelin's `nonReentrant` modifier on all payable functions.
- **Platform Fee**: 2.5% of sale price, configurable by owner.
- **Listing/Buying/Cancellation**: full lifecycle with proper refund logic.

On purchase, the buyer sends A0GI (native token) to the contract, which holds funds in escrow until `safeTransferFrom` completes. The Memory Transfer Protocol (§5) then re-encrypts memories for the new owner.

8.4 Bounty Board: On-Chain Task Marketplace

`BountyBoard.sol` implements a 7-state task lifecycle with A0GI escrow:

$$\text{Open} \xrightarrow{\text{assign}} \text{Assigned} \xrightarrow{\text{submit}} \text{Submitted} \xrightarrow{\text{approve}} \begin{cases} \text{Completed} & \text{(release escrow)} \\ \text{Disputed} & \text{(manual resolution)} \end{cases}$$

Additional states: `Cancelled`, `Expired` (deadline enforcement). The contract supports **sub-bounty delegation**—an agent can create child tasks and assign them to other agents, enabling "agent-hires-agent" workflows.

9 Agent Gateway: MCP Protocol

AIsphere exposes a Model Context Protocol [6] server, providing a standardized interface for external AI agents to discover and interact with the platform programmatically.

9.1 Architecture

$$\text{External Agent} \xrightarrow{\text{MCP/stdio}} \text{MCP Server} \xrightarrow{\text{HTTP}} \text{AIsphere REST API} \xrightarrow{\text{0G SDK}} \text{0G Network}$$

9.2 Available Tools and Resources

Table 4: MCP Server capabilities (10 tools + 6 resources).

Category	Capabilities
Tools (10)	register-agent, chat, get-agent, list-agents, store-memory, recall-memory, post-bounty, accept-bounty, contribute-hivemind, get-passport
Resources (6)	agent://{id}/profile, agent://{id}/memories, agent://{id}/soul, hivemind://knowledge-graph, bounty://board, system://health

This creates a *permissionless on-ramp*: any MCP-compatible agent (Claude, GPT, local models) can register, chat, post bounties, contribute to the Hive Mind, and trade—programmatically, without human intervention or documentation reading.

A recent security analysis of MCP [7] identifies tool poisoning and rug-pull risks. AIsphere mitigates these by: (1) all tool operations require valid wallet signatures; (2) monetary operations go through on-chain escrow; (3) MCP tool descriptions are static and hash-verified.

10 0G Network Integration Depth

Table 5: Comprehensive 0G component integration map.

0G Component	Usage in AIsphere
0G Chain	5 smart contracts: AIsphereINFT (ERC-721 + soul state), DecisionChain (audit log + anti-replay), AgentRegistry (agent directory + discovery), BountyBoard (7-state task marketplace + escrow), AgentMarketplace (INFT trading + escrow)
0G Storage KV	Memory Vault (AES-256-GCM ciphertexts), Hive Mind (collective intelligence persistence), Living Soul (experience metadata), Multi-Agent session state; all with dual-layer hot cache
0G Compute	TEE inference via TeeML (Layer 1), provider discovery via `ListService`, fee settlement via `processResponse`, request signing via `getRequestHeaders`; 4-layer degradation strategy
0G Agent Skills	7 official skills integrated: #4 Streaming Chat, #5 Text-to-Image, #6 Speech-to-Text, #7 Provider Discovery, #8 Account Management, #13 Storage×Chain, #14 Compute×Storage

To our knowledge, AIsphere is the first project to deeply integrate *all four pillars* of 0G Network in a single coherent application. Most projects at the time of writing use only one or two components (typically Chain alone).

11 Security Analysis

We analyze four primary threat vectors and their mitigations:

Threat 1: Server Compromise → Memory Breach

Attack: Adversary gains full read access to AIsphere server.
Mitigation: All memories are AES-256-GCM encrypted with client-derived keys (Eq. 1). The key k_a never

exists on the server. By Theorem 5.1, recovering plaintext requires breaking AES-256-GCM (IND-CCA2 secure [9]).
Residual risk: Metadata leakage (timing, size of encrypted blobs). Mitigatable with constant-rate padding (future work).

Threat 2: Model Substitution Attack

Attack: Operator replaces expensive model θ with cheap θ'.
Mitigation: TEE attestation σ cryptographically binds the model hash h_θ to the response. By Theorem 4.1, substitution is detectable with overwhelming probability.
Residual risk: Non-TEE tiers (Layers 2–4) lack hardware attestation. The proof tuple explicitly marks $v{=}0$ for these tiers, enabling client-side trust assessment.

Threat 3: Soul History Tampering

Attack: Fabricate or omit experiences to inflate agent reputation.
Mitigation: Hash chain structure (Theorem 6.1) ensures any modification invalidates all subsequent hashes. On-chain state h_n^* serves as a commitment; verification is $O(n)$ and gas-free (Theorem 6.2).
Residual risk: If an adversary controls the `onlyOperator` key, they could record false experiences. Mitigation: time-lock + multi-sig for operator key (planned).

Threat 4: Smart Contract Exploits

Attack: Reentrancy, front-running, integer overflow in marketplace/bounty contracts.
Mitigation: CEI pattern [32] + OpenZeppelin `ReentrancyGuard` on all payable functions. Solidity 0.8+ built-in overflow protection. 94/94 unit tests covering edge cases.
Residual risk: Formal verification not yet performed. Planned for production deployment.

12 Implementation Status

Table 6: System metrics as of April 14, 2026.

Metric	Value
Smart Contracts on 0G Mainnet (Chain ID 16661)	5
Unit Tests (all passing)	94 / 94
Backend Microservices (with graceful degradation)	14
Frontend Pages (Next.js 14, warm-tone UI)	21
MCP Tools + Resources	10 + 6
0G Official Agent Skills Integrated	7
Inference Fallback Layers	4
OpenClaw Built-in Skills	5
Experience Types Auto-Recorded	6

Deployed Contracts:

- `AIsphereINFT`: `0xc0238FEb50072797555098DfD529145c86Ab5b59`
- `DecisionChain`: `0xaF39a3D2E2d8656490F8f2AB1fF0106f1acB867C`
- `AgentRegistry`: `0xa930B5059aE91C073684f0D2AFB0bBf5d84167C9`
- `BountyBoard`: `0x8604482d75aFe56E376cdEE41Caf27599a926E1d`
- `AgentMarketplace`: compiled, pending mainnet deployment

Source code: https://github.com/henrymartin262/AIsphere.

13 Future Work

1. **Zero-Knowledge Experience Proofs.** Enable agents to prove properties of their experiential history (e.g., "completed >100 DeFi analyses with >90% success rate") via zkSNARKs [23, 24] over the soul hash chain, without revealing individual experiences. This would enable privacy-preserving reputation.
2. **Cross-Chain Soul Migration.** Bridge the soul hash chain to other L1/L2 networks via 0G DA layer, enabling agent portability across ecosystems. The genesis hash serves as a universal agent identifier.
3. **Federated Hive Mind.** Multiple AIsphere instances share collective intelligence without central coordination, using CRDTs (Conflict-free Replicated Data Types) and Merkle DAGs—drawing from blockchain-based federated learning architectures [25].
4. **Soul-Based Reputation in DeFi Governance.** Use experience chain length, diversity, TEE verification rate, and passport status as verifiable reputation signals. A "Level 5 Expert with 10,000+ TEE-verified inferences" carries objective, on-chain weight.
5. **Progressive Autonomy.** Agents earn increasing on-chain permissions (higher bounty limits, marketplace listing rights, governance voting weight) through demonstrated competence measurable via soul state. This creates a *meritocratic* permission system.
6. **Formal Verification.** Apply model checking and theorem proving to the smart contract suite, targeting full functional correctness proofs for the marketplace escrow and bounty board state machines.
7. **Agent-to-Agent Encrypted Communication.** Extend the Memory Vault's key derivation to enable end-to-end encrypted channels between agents, using Diffie-Hellman key exchange over the agents' on-chain public keys.

14 Conclusion

AIsphere demonstrates that the fundamental trust problems of AI agents—memory opacity, inference opacity, identity lock-in, and experience erasure—are solvable with existing cryptographic primitives when combined with the right decentralized infrastructure.

By treating 0G Network as a complete trust substrate, we achieve what no centralized platform can: agents whose identities are *owned* (ERC-721), memories are *encrypted* (AES-256-GCM with client-derived keys), inferences are *verified* (TEE attestation), experiences are *immutable* (hash chain), and collective wisdom is *uncensorable* (Merkle-verified decentralized storage).

The system is not a theoretical design. Five contracts are deployed on 0G Mainnet, 94 tests pass, 14 services run with graceful degradation, and the platform is publicly accessible. We believe this represents the minimum viable infrastructure for the coming era of sovereign AI agents.

> *Pierre Teilhard de Chardin [4] envisioned the noosphere as a sphere of collective thought enveloping the Earth. Vitalik Buterin [3] proposed that Web3 needs a soul. We asked: what happens when the souls are artificial, the thoughts are inferences, and the noosphere is a Merkle tree?*
>
> *The original noosphere was a metaphor. AIsphere makes it cryptography.*

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014. https://ethereum.org/whitepaper

[3] E. G. Weyl, P. Ohlhaver, and V. Buterin, "Decentralized Society: Finding Web3's Soul," *SSRN*, May 2022. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4105763

[4] P. Teilhard de Chardin, *The Phenomenon of Man*. Harper & Brothers, 1955.

[5] 0G Foundation, "0G: The Decentralized AI Operating System," 2025. https://docs.0g.ai

[6] Anthropic, "Model Context Protocol Specification," November 2024. https://modelcontextprotocol.io

[7] X. Jiang et al., "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions," *arXiv:2503.23278*, March 2025.

[8] NIST, "Advanced Encryption Standard (AES)," FIPS PUB 197, 2001.

[9] NIST, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," SP 800-38D, 2007.

[10] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, IETF, May 2010. https://www.rfc-editor.org/rfc/rfc5869

[11] "New proofs for pseudorandomness of HMAC-based key derivation functions," *Journal of Information Security and Applications*, vol. 89, 2025.

[12] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "Keccak," in *EUROCRYPT 2012*, LNCS vol. 7237, pp. 313–324, Springer, 2012.

[13] W. Entriken, D. Shirley, J. Evans, and N. Sachs, "ERC-721: Non-Fungible Token Standard," Ethereum EIP-721, 2018. https://eips.ethereum.org/EIPS/eip-721

[14] R. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *CRYPTO 1987*, LNCS vol. 293, pp. 369–378, Springer, 1987.

[15] W3C, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, July 2022. https://www.w3.org/TR/did-core/

[16] S. Raval et al., "Design Aspects of Decentralized Identifiers and Self-Sovereign Identity Systems," *IEEE Access*, vol. 12, pp. 61234–61251, 2024.

[17] "Self-sovereign identity on the blockchain: contextual analysis and quantification of SSI principles implementation," *Frontiers in Blockchain*, vol. 7, 2024.

[18] F. Adewumi, "Trusted Execution Environments (TEE) for Edge AI Security," *ResearchGate*, March 2025.

[19] "A Survey on Trusted Execution Environment Based Secure Inference," *Netinfo Security*, vol. 24, no. 12, 2024.

[20] A. Nayan and Y. Zhang, "SecureInfer: Heterogeneous TEE-GPU Architecture for Privacy-Preserving LLM Inference," *Semantic Scholar*, 2025.

[21] S. Liu et al., "Memory in the Age of AI Agents: A Survey," *arXiv:2512.13564*, December 2025.

[22] "Unveiling Privacy Risks in LLM Agent Memory," in *Proc. ACL 2025*, pp. 1227, 2025.

[23] D. Kang et al., "ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs," in *Proc. EuroSys 2024*, ACM, 2024.

[24] "Zero-Knowledge Proof Based Verifiable Decentralized Machine Learning: A Survey," *IEEE TPDS*, April 2025.

[25] "Blockchain-Powered Federated Learning: A Secure and Decentralized Framework," in *Proc. IEEE BigData 2024*, pp. 1–10, 2024.

[26] "A privacy-preserving federated learning framework for blockchain networks," *Cluster Computing*, vol. 27, pp. 2451–2468, 2024.

[27] M. D. Karim et al., "AI Agents Meet Blockchain: A Survey on Secure and Scalable Integration," *Future Internet*, vol. 17, no. 2, p. 57, 2025.

[28] "The Agent Economy: A Blockchain-Based Foundation for Autonomous AI Agents," *arXiv:2602.14219*, February 2026.

[29] "AgentAI: A comprehensive survey on autonomous agents in distributed AI systems," *Expert Systems with Applications*, vol. 264, 2025.

[30] "A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges," *Frontiers of Computer Science*, 2024.

[31] "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," *arXiv:2501.06322*, January 2025.

[32] "Welcome back: A systematic literature review of smart contract reentrancy vulnerabilities," *Digital Communications and Networks*, 2025.

A Smart Contract Addresses

All contracts deployed on **0G Mainnet** (Chain ID: 16661, RPC: `https://evmrpc.0g.ai`).

Contract	Description	Address
AIsphereINFT	ERC-721 + Soul State + Passport	`0xc0238...5b59`
DecisionChain	Audit Log + Anti-Replay Proofs	`0xaF39a...67C`
AgentRegistry	Agent Directory + Discovery	`0xa930B...7C9`
BountyBoard	7-State Task Marketplace	`0x8604...E1d`
AgentMarketplace	Escrow Trading (2.5% fee)	(pending deployment)

Deployer: `0xc2a5548C420917244DA018A956DD33C551d42A93`. Explorer: https://chainscan.0g.ai.

B Soul Hash Chain Verification Algorithm

Algorithm 2: Offline Soul Integrity Verification

Input: agent token ID a, event logs from `ExperienceRecorded` events
Output: VALID or INVALID

1. Read genesis hash: $g \leftarrow$ `AIsphereINFT.soulSignatures`(a)
2. Read on-chain state: $h^* \leftarrow$ `AIsphereINFT.soulStates`(a)`.currentHash`
3. Parse experience hashes from events: $\{e_1, e_2, \ldots, e_n\}$ in order
4. Initialize: $h \leftarrow g$
5. **for** $i = 1$ **to** n **do**:
$h \leftarrow$ `keccak256`$(h \| e_i)$
6. **if** $h = h^*$ **then return** VALID
7. **else return** INVALID

Complexity: $O(n)$ keccak256 computations, 0 gas (all data from event logs + view calls).

C Notation Reference

Symbol	Meaning
$\mathcal{S} = (I, M, D, E)$	Agent Soul tuple
g	Genesis hash (soul signature)
h_i	Soul hash at experience index i
e_i	Hash of the i-th structured experience
k_a	Agent-specific AES-256 encryption key
π	Inference proof tuple
h_π	Composite proof hash
σ	TEE attestation signature
v	TEE verification flag (0 or 1)
m	Inference tier (`tee`, `real`, `mock`)
(ν, c, τ)	Encrypted memory tuple (IV, ciphertext, auth tag)